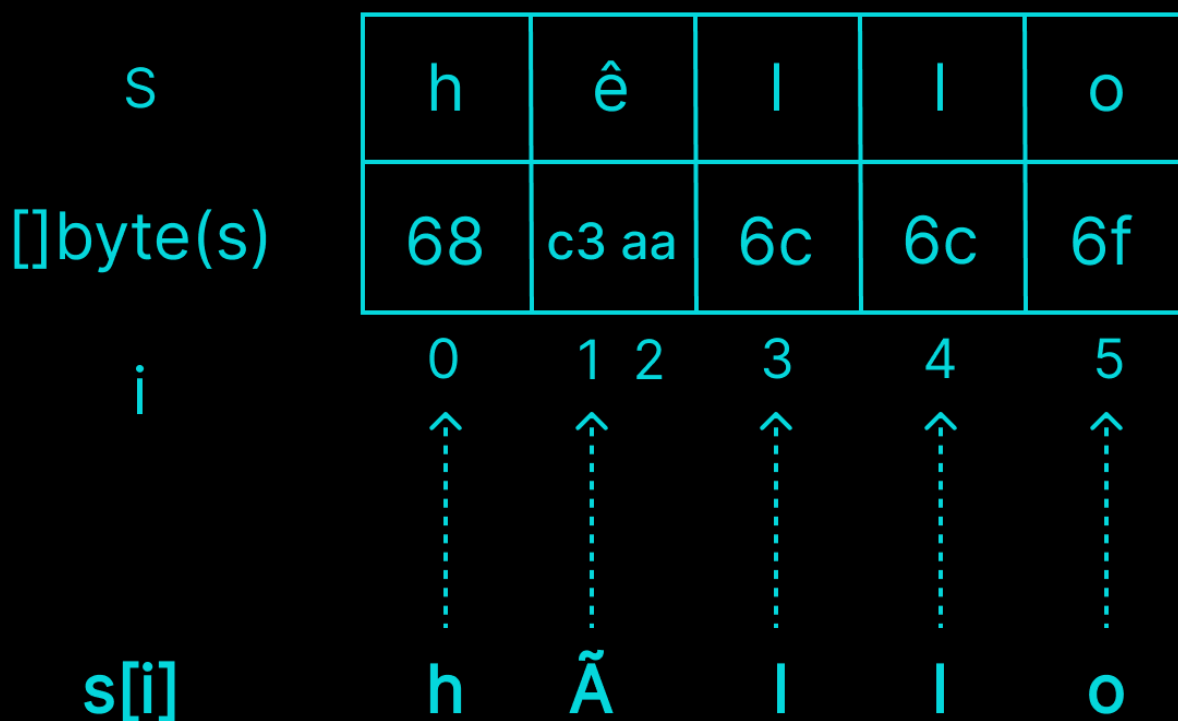




Before start, make sure you've read the following materials:

- [Strings, bytes, runes and characters in Go](#)
- [Understanding Strings, Bytes, and Runes in Golang](#)
- [Strings in Golang](#)
- [Tour of Go: Maps](#)
- [Maps in Go](#)
- [Maps interview questions in Go](#)
- [YouTube: Maps in Go: A Complete Guide | Golang for Devs](#)

Strings, bytes, and runes

🔗 Important

Golang has no dedicated `char` type. It uses `rune` and `byte` to represent character values. Strings,

being read-only slices of bytes, yield individual bytes when indexed.

What is a string?

Let's start with some basics.

Tip

In Go, a string is in effect a read-only slice of bytes.

Example:

```
s := "Hello, 世界" // "Hello, 世界"
fmt.Println(s)
```

Key Points:

- Strings are immutable.
- Strings support UTF-8 encoding.
- The `len()` function returns the number of bytes, not characters.

```
s := "hello"
fmt.Println(len(s)) // 5
```

| What will be the output here then? Why?

```
s := "世界"
fmt.Println(len(s))
```

If you need to find out how many actual characters are in a string, use the `unicode/utf8` package:

```
utf8.RuneCountInString(s)
```

Runes

To work correctly with Unicode in Go, you need to understand what a **rune** is. It is an alias for **int32** and represents a Unicode code point—that is, a specific Unicode character.

 Note

There is a huge difference between a rune and a byte. A single character can take up anywhere from 1 to 4 bytes. Therefore, when working with text containing non-English characters, this cannot be ignored.

 Caution

Incorrect string iteration!

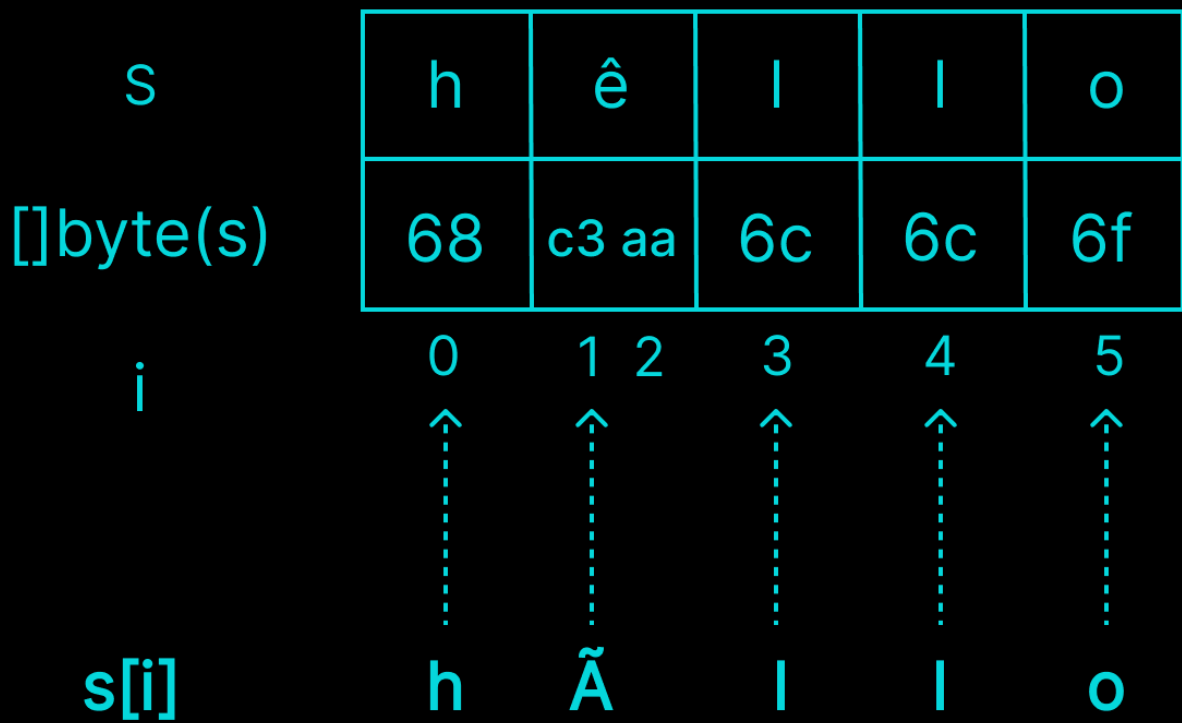
```
s := "hêllo"
for i := range s {
    fmt.Printf("position %d: %c\n", i, s[i])
}
```

Instead of getting the expected nice symbols, you end up with something like this:

```
position 0: h
position 1: Ã
position 3: l
...
```

 Note

Why is that? Because `s[i]` returns a byte, not a rune. And the character `ê` takes up two bytes. Such a loop iterates over byte indices, not rune indices.



🔗 Important

Correct approach:

```
for i, r := range s {  
    fmt.Printf("position %d: %c\n", i, r)  
}
```

🔗 Question

How does Go represent Unicode characters internally, and why does indexing a string like `s[i]` yield a byte rather than a character code point?

Now `r` is the rune itself, and the output will be correct.

🔗 Question

How can you access the second rune in a string in a readable form? And how does it work?

Trim Functions

The functions `TrimRight`, `TrimLeft`, `TrimPrefix`, and `TrimSuffix` can be confusing. For example:

```
strings.TrimRight("123oxo", "xo") // Returns "123"
```

You might mistakenly expect `"123o"`, but `TrimRight` removes all trailing characters found in the set `"xo"`.

If you need to remove a specific suffix, use `TrimSuffix`:

```
strings.TrimSuffix("123oxo", "xo") // Now returns "123o"
```

The same applies to prefixes: `TrimPrefix` removes a specific substring from the beginning.

Tip

On the defence be prepared to demonstrate the usage of `TrimRight`, `TrimLeft`, `TrimPrefix`, and `TrimSuffix` functions.

Slow string concatenation

Caution

Suppose there is a list of strings that need to be concatenated. An incorrect solution:

```
func concat(values []string) string {
    s := ""
    for _, v := range values {
        s += v
    }
    return s
}
```

The problem here is that every `+=` creates a new string. This is costly, especially if there are many strings.



Tip

The best approach is to use `strings.Builder`:

```
func concat(values []string) string {
    var sb strings.Builder
    for _, v := range values {
        sb.WriteString(v)
    }
    return sb.String()
}
```

If the size of the result is known in advance, you can use `sb.Grow(totalLength)` to speed up the process.



Task

Demonstrate the concatenation of strings using `strings.Builder`'s `Grow` method.

Unnecessary conversions between `[]byte` and `string`

Many are used to working with strings, but in Go, most I/O operations work with `[]byte`. Converting back and forth adds unnecessary overhead.

Instead of:

```
s := string(bytes.TrimSpace([]byte(data)))
```

It is recommended to work directly with bytes:

```
b := bytes.TrimSpace(data)
```

The `bytes` package contains equivalents of all popular functions from `strings`: `Split`, `Contains`, `Index`, `TrimSpace`, etc. .



Task

```
func processPayload(data []byte) string {  
    // Unnecessary type conversions  
    trimmed := string(bytes.TrimSpace(data))  
    clean := strings.ReplaceAll(trimmed, "\r", "")  
    return clean  
}
```

Requirements:

- Explain what happens under the hood in terms of memory allocation when converting between `[]byte` and `string`.
 - Refactor the `processPayload` function to work entirely with byte slices using the standard `bytes` package, eliminating unnecessary type conversions.
 - Write a Go benchmark test (`BenchmarkProcessPayload`) comparing the original and optimized versions, proving the reduction in allocations (`allocs/op`) and memory footprint (`B/op`).
-

Key takeaways:

- A string is a sequence of bytes.
 - A rune is a Unicode character.
 - Use `range` for correct iteration.
 - The difference between `TrimRight` and `TrimSuffix`.
 - Use `strings.Builder` for concatenation.
 - Avoid unnecessary conversions between `[]byte` and `string`.
 - Create copies of substrings to prevent memory leaks(will be discovered in the next weeks).
-

Maps

Map operations

To perform read, update, and delete operations, Go first computes the hash of the key to locate the corresponding bucket. It then searches for the appropriate slot within the bucket using control words.

```
// Example of map operations
m := map[string]int{"one": 1, "two": 2}
fmt.Println(m["one"]) // Reading a value
m["two"] = 20         // Updating a value
delete(m, "one")      // Deleting a value
```

🔍 Question

How to test whether a key is present with a two-value assignment?

💡 Important

On average, operations are performed in $O(1)$ time thanks to an efficient **hash function** and the **Swiss Tables** structure. Even in the worst case, performance remains high due to optimized probing algorithms.

Initializing Maps in Go

Maps in Go offer great flexibility for storing data as key-value pairs. However, their performance and memory usage depend heavily on how they are initialized. To understand why proper map initialization is so important, let's examine the basics of how they work and how they grow.

Issues with Inefficient Initialization

Suppose we create a map with three elements:

```
m := map[string]int{
    "1": 1,
    "2": 2,
    "3": 3,
}
```

Under the hood, this map uses a **hash table** with a small number of buckets to store these three elements. However, what happens

if we add a million elements? To avoid issues, the map automatically increases in size, adding new `buckets` as needed.

Load factor: The average number of elements per bucket exceeds a threshold (currently 6.5 in the Go implementation). This parameter is internal and may change in future versions.

Bucket overflow: Too many buckets contain more than `8` elements.

When the map grows, all keys are redistributed across the new buckets. In the worst-case scenario, inserting a new key can take $O(n)$ time, where n is the total number of elements in the map.

Optimizing map initialization

If the number of elements to be added to the map is known in advance, you can use the `make` function and specify the initial size:

```
m := make(map[string]int, 1_000_000)
```

Task

You are given a map storing student exam scores:

```
scores := map[string]int{
    "Alice": 90,
    "Bob":   0, // Bob took the exam but scored 0
}
```

If you read `scores["Charlie"]`, Go returns `0` (the default zero-value for `int`), which makes it look like Charlie scored 0 just like Bob.

Instructions:

1. Use the two-value assignment (`value, ok := map[key]`) to check whether `"Bob"` and `"Charlie"` exist in the `scores` map.
 2. Print `"Bob is in the map"` if `ok` is `true`, otherwise print `"Bob is missing"`.
 3. Do the same check for `"Charlie"`.
-

Task

Create an inventory management program for a small store.

Instructions:

1. Initialize a map named `inventory` with the following key-value pairs (`string` -> `int`):

```
- ` "apples" `: `10`  
- ` "bananas" `: `5`
```

2. Read: Print the current stock of `"apples"`.
 3. Update: Increase the number of `"bananas"` to `12`.
 4. Insert: Add a new item `"oranges"` with a quantity of `8`.
 5. Delete: Remove `"apples"` from the inventory using the `delete()` built-in function.
 6. Print the updated map to confirm the changes.
-

Task

Solve the problem from the tour of Go -> Exercise: Maps

